Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №2
Варіант 9
На тему
“Функції вищого порядку, lambda, композиція”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- опанувати функції вищого порядку map, filter та reduce;
- навчитися використовувати lambda-вирази у доречних місцях;
- реалізувати композицію функцій і зібрати з неї конвеєр;
- побудувати конвеєр перетворень над списком записів без мутацій.

Теоретичні відомості:

- Функція вищого порядку приймає або повертає інші функції: map, filter, sorted з key, functools.reduce.
- lambda - анонімна функція з одного виразу, зручна для коротких предикатів і ключів сортування.
- Композиція compose(f, g)(x) = f(g(x)) дозволяє збирати складну обробку з маленьких чистих кроків.
- reduce згортає послідовність в одне значення за допомогою функції-акумулятора та початкового значення.

Завдання (варіант 9 - Працівники компанії):

Записи працівників: id, name, department, monthly_hours. Обчислити сумарні години, застосувати коефіцієнт відділу, відфільтрувати за мінімумом годин і сформувати Top-N. Агрегація reduce: count, total_hours, avg_hours.

Хід роботи

Проєкт оформлено за структурою, погодженою з викладачем для ЛР1: core.py з чистими функціями, app.py з введенням-виведенням і CLI, main.py як точка входу, тести в tests/test_core.py, звіт у папці report.

Запис працівника описано через TypedDict Employee. Кожен етап конвеєра - функція вищого порядку, яка повертає перетворення Stage над ітерованим набором записів: normalize_names і with_total_hours використовують map, boost_department множить години вибраного відділу на коефіцієнт, at_least_hours відбирає записи через filter з lambda-предикатом, sort_by_hours_desc сортує за двома ключами (години спадно, id зростанням), take бере Top-N через itertools.islice.

Функція compose реалізована через functools.reduce і застосовує етапи справа наліво, pipe - зліва направо. build_pipeline збирає з них повний конвеєр. Кожен етап повертає нові словники, тому вхідні записи не змінюються.

reduce_stats однією згорткою reduce обчислює count, total_hours та avg_hours. app.py розбирає аргументи командного рядка (--top, --boost-dept, --factor, --min-hours, --data) і друкує рейтинг та статистику.

Тести перевіряють Top-N, відсутність мутацій, правильність агрегації, порядок композиції, тай-брейкер сортування, коефіцієнт відділу, фільтр і граничні випадки.

Файл core.py

core.py рядки 1-61

```python
"""Чисте ядро ЛР2, варіант 9: конвеєр обробки працівників компанії."""

from collections.abc import Callable, Iterable
from functools import reduce
from itertools import islice
from typing import TypedDict


class Employee(TypedDict, total=False):
    """Запис працівника. Поле total_hours з'являється після обробки."""

    id: int
    name: str
    department: str
    monthly_hours: list[float]
    total_hours: float


class Stats(TypedDict):
    """Результат reduce-агрегації."""

    count: int
    total_hours: float
    avg_hours: float


Stage = Callable[[Iterable[Employee]], Iterable[Employee]]


def compose(*funcs: Stage) -> Stage:
    """Композиція справа наліво: compose(f, g)(x) == f(g(x))."""

    def composed(records: Iterable[Employee]) -> Iterable[Employee]:
        """Пропускає записи через усі етапи, починаючи з останнього."""
        return reduce(lambda acc, f: f(acc), reversed(funcs), records)

    return composed


def pipe(*funcs: Stage) -> Stage:
    """Композиція зліва направо: pipe(f, g)(x) == g(f(x))."""
    return compose(*reversed(funcs))


def normalize_names() -> Stage:
    """Приводить імена до вигляду Title Case."""

    def normalize(e: Employee) -> Employee:
        """Повертає новий запис з нормалізованим ім'ям."""
        return {**e, "name": str(e.get("name", "")).strip().title()}

    return lambda recs: map(normalize, recs)


def with_total_hours() -> Stage:
    """Додає поле total_hours як суму відпрацьованих годин за місяці."""

    def add_total(e: Employee) -> Employee:
        """Повертає новий запис із сумою годин."""
        return {**e, "total_hours": float(sum(e.get("monthly_hours", [])))}
```

core.py рядки 62-108

```python
    return lambda recs: map(add_total, recs)


def boost_department(department: str, factor: float) -> Stage:
    """Множить total_hours на коефіцієнт для вибраного відділу."""

    def boost(e: Employee) -> Employee:
        """Повертає новий запис з урахуванням коефіцієнта відділу."""
        hours = e.get("total_hours", 0.0)
        if e.get("department") == department:
            return {**e, "total_hours": hours * factor}
        return {**e, "total_hours": hours}

    return lambda recs: map(boost, recs)


def at_least_hours(min_hours: float) -> Stage:
    """Відбирає працівників, у яких total_hours не менше за мінімум."""
    return lambda recs: filter(lambda e: e.get("total_hours", 0.0) >= min_hours, recs)


def sort_by_hours_desc() -> Stage:
    """Сортує за total_hours спадно, при рівності за id зростанням."""
    return lambda recs: sorted(
        recs, key=lambda e: (-e.get("total_hours", 0.0), e.get("id", 0))
    )


def take(n: int) -> Stage:
    """Бере перші n записів."""
    return lambda recs: list(islice(recs, max(n, 0)))


def reduce_stats(records: Iterable[Employee]) -> Stats:
    """Обчислює count, total_hours та avg_hours однією згорткою reduce."""
    count, total = reduce(
        lambda acc, e: (acc[0] + 1, acc[1] + e.get("total_hours", 0.0)),
        records,
        (0, 0.0),
    )
    return {
        "count": count,
        "total_hours": total,
        "avg_hours": total / count if count else 0.0,
    }
```

core.py рядки 109-124

```python
def build_pipeline(
    *,
    top_n: int = 3,
    department: str = "IT",
    factor: float = 1.1,
    min_hours: float = 400.0,
) -> Stage:
    """Збирає конвеєр через compose: нормалізація, сума, коефіцієнт, фільтр, Top-N."""
    return compose(
        take(top_n),
        sort_by_hours_desc(),
        at_least_hours(min_hours),
        boost_department(department, factor),
        with_total_hours(),
        normalize_names(),
    )
```

Файл app.py

app.py рядки 1-59

```python
"""Оболонка ЛР2 (I/O та CLI): читання аргументів, даних і друк результатів."""

import argparse
import json
from collections.abc import Sequence
from pathlib import Path

from core import Employee, build_pipeline, reduce_stats, with_total_hours


def sample_employees() -> list[Employee]:
    """Демонстраційні дані працівників."""
    return [
        {
            "id": 1,
            "name": "ivan petrenko",
            "department": "IT",
            "monthly_hours": [160, 168, 172],
        },
        {
            "id": 2,
            "name": "olena koval",
            "department": "HR",
            "monthly_hours": [150, 140, 155],
        },
        {
            "id": 3,
            "name": "andrii melnyk",
            "department": "IT",
            "monthly_hours": [120, 130, 125],
        },
        {
            "id": 4,
            "name": "maria shevchenko",
            "department": "Sales",
            "monthly_hours": [176, 180, 170],
        },
        {
            "id": 5,
            "name": "taras bondar",
            "department": "Sales",
            "monthly_hours": [100, 90, 95],
        },
        {
            "id": 6,
            "name": "iryna lysenko",
            "department": "IT",
            "monthly_hours": [158, 162, 160],
        },
    ]


def load_employees(path: Path) -> list[Employee]:
    """Читає список працівників із JSON-файлу."""
    with path.open(encoding="utf-8") as source:
        data: list[Employee] = json.load(source)
    return data
```

app.py рядки 60-106

```python
def parse_args(argv: Sequence[str] | None) -> argparse.Namespace:
    """Розбирає аргументи командного рядка."""
    parser = argparse.ArgumentParser(description="Top-N працівників за годинами")
    parser.add_argument("--data", type=Path, default=None, help="JSON з даними")
    parser.add_argument("--top", type=int, default=3, help="кількість записів")
    parser.add_argument("--boost-dept", default="IT", help="відділ з коефіцієнтом")
    parser.add_argument("--factor", type=float, default=1.1, help="коефіцієнт")
    parser.add_argument("--min-hours", type=float, default=400.0, help="мінімум годин")
    return parser.parse_args(argv)


def render(top: Sequence[Employee], employees: Sequence[Employee]) -> None:
    """Друкує рейтинг і статистику."""
    print(f"{'ID':>3} {'Ім`я':<18} {'Відділ':<7} {'Години':>8}")
    for e in top:
        print(
            f"{e['id']:>3} {e['name']:<18} {e['department']:<7} "
            f"{e['total_hours']:>8.1f}"
        )
    stats = reduce_stats(with_total_hours()(employees))
    print(
        f"\nВсі працівники: count={stats['count']}, "
        f"total_hours={stats['total_hours']:.1f}, avg_hours={stats['avg_hours']:.1f}"
    )
    top_stats = reduce_stats(top)
    print(
        f"Top-N: count={top_stats['count']}, "
        f"total_hours={top_stats['total_hours']:.1f}, "
        f"avg_hours={top_stats['avg_hours']:.1f}"
    )


def run(argv: Sequence[str] | None = None) -> None:
    """Запускає конвеєр з параметрами з командного рядка."""
    args = parse_args(argv)
    employees = load_employees(args.data) if args.data else sample_employees()
    pipeline = build_pipeline(
        top_n=args.top,
        department=args.boost_dept,
        factor=args.factor,
        min_hours=args.min_hours,
    )
    render(list(pipeline(employees)), employees)


if __name__ == "__main__":
    run()
```

Файл main.py

main.py рядки 1-12

```python
"""Точка входу лабораторної роботи №2, варіант 9."""

from app import run


def main() -> None:
    """Запуск програми аналізу годин працівників компанії."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_core.py

test_core.py рядки 1-60

```python
"""pytest-тести ЛР2, варіант 9."""

from copy import deepcopy

import pytest

from core import (
    Employee,
    at_least_hours,
    boost_department,
    build_pipeline,
    compose,
    normalize_names,
    pipe,
    reduce_stats,
    sort_by_hours_desc,
    take,
    with_total_hours,
)


@pytest.fixture
def employees() -> list[Employee]:
    """Тестові записи працівників."""
    return [
        {"id": 1, "name": " ivan ", "department": "IT", "monthly_hours": [100, 100]},
        {"id": 2, "name": "olena", "department": "HR", "monthly_hours": [150, 150]},
        {"id": 3, "name": "petro", "department": "IT", "monthly_hours": [50, 50]},
        {"id": 4, "name": "maria", "department": "HR", "monthly_hours": [100, 100]},
    ]


def test_top_n(employees: list[Employee]) -> None:
    """Тест: Top-N з коефіцієнтом відділу і фільтром."""
    pipeline = build_pipeline(top_n=2, department="IT", factor=1.5, min_hours=150)
    result = list(pipeline(employees))
    assert [e["id"] for e in result] == [1, 2]
    assert result[0]["total_hours"] == pytest.approx(300.0)
    assert result[0]["name"] == "Ivan"


def test_no_mutation(employees: list[Employee]) -> None:
    """Тест: вхідні записи не змінюються."""
    original = deepcopy(employees)
    list(build_pipeline(top_n=3)(employees))
    assert employees == original
    assert all("total_hours" not in e for e in employees)


def test_reduce_stats(employees: list[Employee]) -> None:
    """Тест: правильність reduce-агрегації."""
    stats = reduce_stats(with_total_hours()(employees))
    assert stats == {"count": 4, "total_hours": 800.0, "avg_hours": 200.0}


def test_reduce_stats_empty() -> None:
    """Тест: агрегація порожнього списку."""
    assert reduce_stats([]) == {"count": 0, "total_hours": 0.0, "avg_hours": 0.0}
```

test_core.py рядки 61-110

```python
def test_compose_order() -> None:
    """Тест: compose застосовує функції справа наліво, pipe зліва направо."""
    first = take(1)
    ordered = sort_by_hours_desc()
    data: list[Employee] = [
        {"id": 1, "total_hours": 10.0},
        {"id": 2, "total_hours": 20.0},
    ]
    assert [e["id"] for e in compose(first, ordered)(data)] == [2]
    assert [e["id"] for e in pipe(ordered, first)(data)] == [2]
    assert [e["id"] for e in compose(ordered, first)(data)] == [1]


def test_sort_tie_breaker() -> None:
    """Тест: при рівних годинах сортування за id зростанням."""
    data: list[Employee] = [
        {"id": 3, "total_hours": 5.0},
        {"id": 1, "total_hours": 5.0},
        {"id": 2, "total_hours": 9.0},
    ]
    assert [e["id"] for e in sort_by_hours_desc()(data)] == [2, 1, 3]


def test_boost_only_selected_department(employees: list[Employee]) -> None:
    """Тест: коефіцієнт застосовується лише до вибраного відділу."""
    stage = compose(boost_department("HR", 2.0), with_total_hours())
    hours = {e["id"]: e["total_hours"] for e in stage(employees)}
    assert hours == {1: 200.0, 2: 600.0, 3: 100.0, 4: 400.0}


def test_filter_min_hours(employees: list[Employee]) -> None:
    """Тест: фільтр за мінімальною кількістю годин."""
    stage = compose(at_least_hours(200), with_total_hours())
    assert [e["id"] for e in stage(employees)] == [1, 2, 4]


def test_normalize_names(employees: list[Employee]) -> None:
    """Тест: нормалізація імен."""
    assert [e["name"] for e in normalize_names()(employees)] == [
        "Ivan",
        "Olena",
        "Petro",
        "Maria",
    ]


def test_take_zero_and_negative(employees: list[Employee]) -> None:
    """Тест: take(0) і від'ємне n дають порожній список."""
    assert take(0)(employees) == []
    assert take(-1)(employees) == []
```

Запуск програми

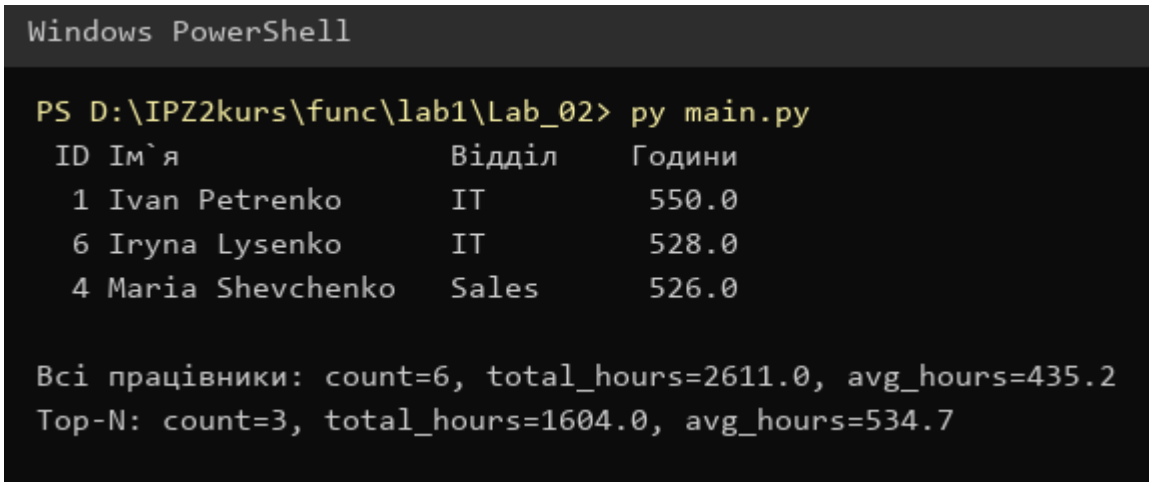


Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_02> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_02
plugins: platformdirs-4.12.4
collecting ... collected 10 items

tests/test_core.py::test_top_n PASSED                                    [ 10%]
tests/test_core.py::test_no_mutation PASSED                              [ 20%]
tests/test_core.py::test_reduce_stats PASSED                             [ 30%]
tests/test_core.py::test_reduce_stats_empty PASSED                       [ 40%]
tests/test_core.py::test_compose_order PASSED                            [ 50%]
tests/test_core.py::test_sort_tie_breaker PASSED                         [ 60%]
tests/test_core.py::test_boost_only_selected_department PASSED           [ 70%]
tests/test_core.py::test_filter_min_hours PASSED                         [ 80%]
tests/test_core.py::test_normalize_names PASSED                          [ 90%]
tests/test_core.py::test_take_zero_and_negative PASSED                   [100%]

============================== 10 passed in 0.04s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_02> py -m mypy --strict core.py app.py main.py tests/test_core.py
Success: no issues found in 4 source files
PS D:\IPZ2kurs\func\lab1\Lab_02> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_02> py -m black --check .
All done!
4 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи побудовано функціональний конвеєр обробки працівників на основі map, filter, sorted та reduce. Композиція compose дозволила зібрати конвеєр з маленьких чистих етапів, кожен з яких легко замінити чи переставити. Lambda-вирази використано для предикатів, ключів сортування та акумулятора reduce. Вхідні дані не змінюються, що підтверджено тестами, а код проходить mypy, black і ruff.